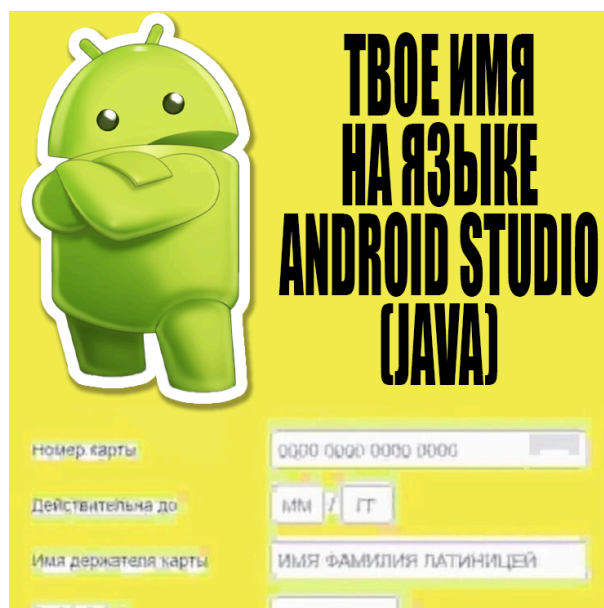


Лабораторная работа №16

Реализация гетерогенного ListView

Основные правила

1. Не бойтесь задавать любые вопросы
2. Если хотите сделать что-то свое, то предлагайте – будем думать
3. Советую читать текст, тут написано много полезного
- 4.



О чем лабораторная?

На прошлом занятии вы узнали, что такое адаптеры и как они связывают данные с элементами списка. Сегодня применим эти знания на практике: создадим список с разными типами элементов.

Главное, что нужно понять

- **Гетерогенный список** - список, в котором элементы выглядят по-разному. Например, некоторые элементы содержат только текст, другие - текст и изображение, третьи - заголовок и описание
- Адаптер должен уметь возвращать **разные типы элементов** для разных позиций

Часть 1. Самостоятельная работа

Задание

Выберите одну из тем или предложите свою:

1. **Список сообщений чата** - входящие сообщения (текст + имя отправителя) и исходящие сообщения (только текст)
2. **Список товаров** - обычные товары (название + цена) и товары со скидкой (название + цена + старая цена)

3. **Лента новостей** - обычные новости (заголовок + текст) и важные новости (заголовок + текст + изображение)
4. **Своя тема** - любой список с двумя типами элементов

Требования (если что-то забыли нажмите сюда)

- Создайте модель данных с полем `type` и константами `TYPE_1`, `TYPE_2`
- Создайте два макета для разных типов элементов
- Создайте адаптер, наследующийся от `BaseAdapter`
- Переопределите `getViewTypeCount()` и `getItemViewType(int position)`
- В `getView` создавайте разные макеты в зависимости от типа
- Все строки вынесите в ресурсы (`strings.xml`)

Подсказки

Модель данных:

```
public class Message {
    public static final int TYPE_INCOMING = 0; // константа для первого типа
    public static final int TYPE_OUTGOING = 1; // константа для второго типа

    private String text;
    private String sender;
    private int type; // поле для хранения типа

    public Message(String text, String sender, int type) {
        this.text = text;
        this.sender = sender;
        this.type = type;
    }

    public String getText() { return text; }
    public String getSender() { return sender; }
    public int getType() { return type; }
}
```

Метод getView():

```
@Override
public View getView(int position, View convertView, ViewGroup parent) {
    // Ваша задача:
    // 1. Если convertView == null, создайте нужный макет в зависимости от типа
    if (convertView == null) {
        LayoutInflater inflater = LayoutInflater.from(context);

        // смотрим какой тип
        if (type == Message.TYPE_INCOMING) {
            // если входящее - используем ОДИН макет
        } else {
            // если исходящее - используем ДРУГОЙ макет
        }
    }
}
```

```
    }  
}  
  
// 2. Заполните данными так же в зависимости от типа  
// ...  
  
return convertView;  
}
```

Требования к итоговой версии

1. Приложение отображает список с разными типами элементов
2. Адаптер корректно определяет тип элемента и использует нужный макет
3. Все текстовые данные вынесены в `strings.xml`